



Fundamentals of Programming: Pointers in C++

Bilal Ahmed

```
types.Operator):  
    X mirror to the selected  
    object.mirror_mirror_x"  
    mirror X"
```

Agenda

- Address Operator
- Definition of a pointer
- Syntax of a pointer
- Creating pointer variables and accessing those
- Use of Dereference Operator
- Pointer to Pointer
- Pointer Arithmetic (Increment/Decrement)

Address Operator (&)

- ▶ Address operator returns the memory address of a variable.

```
#include<iostream>
using namespace std;

int main(){
int a=5;
cout<<&a<<endl;
return 0;
}
```

```
C:\Users\92306\Desktop\Aror  X + v
0x70fe1c
-----
Process exited after 0.06724 seconds with return value 0
Press any key to continue . . . |
```

```
#include<iostream>
using namespace std;

int main(){
int a=5;
int b=&a;
cout<<b<<endl;
return 0;
}
```

Can we
do this....

01

y is declared as int, but &x is an **address**, so it should be stored in a pointer type.

02

Assigning a memory address to an int is **invalid and causes a type mismatch**.

03

Printing y with cout will try to display it as an integer value, not as a pointer address.

Pointer

6



A variable which holds the memory address of other variables of same data type.



Pointers are symbolic representation of memory addresses.

Syntax

Syntax:

```
datatype *var_name;  
int *ptr;    // ptr can point to an address which holds int data
```

Pointer variable returns the address where variable a is stored in memory

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    int x=5;
    int* y = &x;
    cout << y;
}
```


Dereference operator (*)

IT GIVES THE VALUE STORED AT ANY MEMORY ADDRESS.

Accessing Values

10

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    int x=5;
    int* y = &x;
    cout << *y;
}
```

Terminology

11

& (Address-of operator) →
Returns the **memory address** of a variable.

*** (Dereference operator) →**
Accesses the **value stored at a pointer's address.**

Difference

12

int y; → y is a **normal integer variable** that stores an **actual numeric value** (like 5, 10, 20).

int* y; → y is a **pointer**, not a number; it stores an **address** of an integer variable in memory.

Summary

13

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int *ptr = &x; // pointer storing address of x

    cout << "Value of x: " << x << endl;
    cout << "Address of x: " << &x << endl;
    cout << "Pointer ptr stores: " << ptr << endl;
    cout << "Value pointed by ptr: " << *ptr << endl;

    return 0;
}
```

Another program, to illustrate the usage of dereference operator *

14

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    string x="Bilal";
    string* y = &x;
    cout << *y;
}
```

Variety of uses...



Call by reference



Iterating over arrays



Manipulating dynamic arrays

Why to associate data type with a pointer, Why not directly store address

- ▶ Data type tells us how many bytes the data is stored in.
- ▶ When we increment a pointer, we increase the pointer by the size of data type it points to.

Pointer to Pointer

```
int main(){
string a="Hello";
string* b=&a;
string**c =&b;

cout<<&a<<endl;
cout<<b<<endl;
cout<<*b<<endl;
cout<<&b<<endl;

cout<<c<<endl;
cout<<*c<<endl;
cout<<**c;
```

```
C:\Users\92306\Desktop\Aror
0x71fdf0
0x71fdf0
Hello
0x71fde8
0x71fde8
0x71fdf0
Hello
-----
Process exited after 0.
Press any key to continue
```

Pointer Arithmetic (Increment/Decrement)

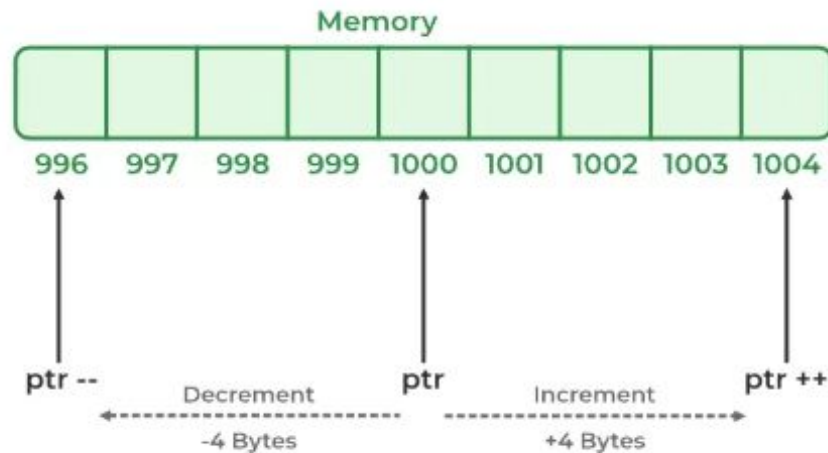
18

- ▶ Applying increment/decrement operator on pointer variable will change the memory address

Pointer Arithmetic (Addition)

19

Pointer Increment & Decrement



Increment in Pointer

20

```
#include<iostream>
using namespace std;

int main(){

    int a=5;
    int* ptr=&a;
    cout<<ptr<<endl;
    cout<<++ptr<<endl;
    return 0;
}
```

```
C:\Users\92306\Desktop\Aror  ×  +  ▾
0x70fe14
0x70fe18

-----
Process exited after 0.08117 seconds with return value 0
Press any key to continue . . .
```

What about this:

21

```
#include<iostream>
using namespace std;

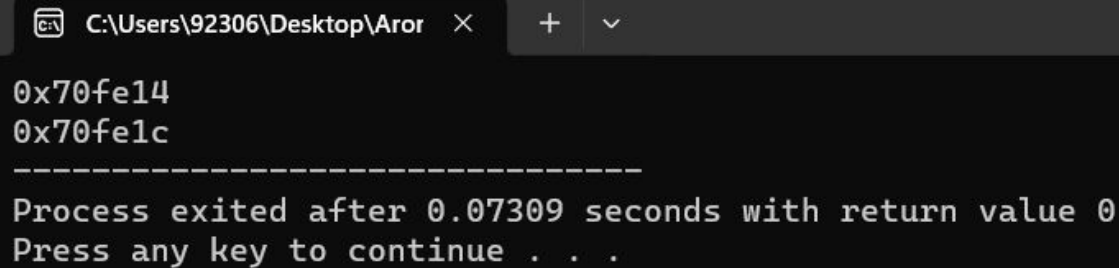
int main(){
int a=5;
int* ptr=&a;
cout<<ptr<<endl;
cout<<++ptr<<endl;
cout<<++ptr;
return 0;
}
```


Further on Increment:

22

```
#include<iostream>
using namespace std;

int main(){
int a=5;
int* ptr=&a;
cout<<ptr<<endl;
cout<<ptr+2;
}
```



```
C:\Users\92306\Desktop\Aror x + v
0x70fe14
0x70fe1c
-----
Process exited after 0.07309 seconds with return value 0
Press any key to continue . . .
```

Pointer Decrement

23

```
#include<iostream>
using namespace std;

int main(){
int a=5;
int* ptr=&a;
cout<<ptr<<endl;
cout<<--ptr;
}
```

```
C:\Users\92306\Desktop\Aror  ×  +  ▾
0x70fe14
0x70fe10
-----
Process exited after 0.06461 seconds with return value 0
Press any key to continue . . .
```

Null Pointer

24

- ▶ A Pointer which points no where
- ▶ If we don't have address to be assigned to a pointer, then we can simply use NULL.
- ▶ Two ways to assign a pointer as null:
 - ▶ `int *ptr1=0;`
 - ▶ `int *ptr2=NULL;`

Null vs Uninitialized Pointer

25

- ▶ An uninitialized pointer stores an undefined value.
- ▶ A null pointer stores a defined value, but one that is defined by the environment to not be a valid address for any member or object.